14.4 Go-live em produção — status de entrega

Bloco 14.4 Go-live em produção · Sprint 14 — Migração da planilha e go-live · Data: 2026-10-26 · Branch: `main` · Ambiente: gc-dev (`gzbmhgnpoehormnidmgg`), como ensaio da virada

Resumo

Ensaio da virada executado em gc-dev em 2026-10-26 (21:55 a 22:01), com os números reais abaixo. O Breno não tem acesso ao gc-prod e decidiu em 26/10: "não vamos fazer nada em gc-prod, finalize tudo pelo dev". O bloco virou o ensaio do go-live em gc-dev: backup e teste de restauração, conferência do histórico de migrations, importação da planilha falsa com `--ate 2026-11-12`, conciliação, segunda gravação, desfazer e fumaça. Nenhuma atividade tocou o gc-prod. O que só existe em prod fica como registro para quem for operar. O procedimento completo está em `docs/tecnicos/runbook-producao.md`.

gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod.

O gc-prod continua pausado. Conferido em 2026-10-26 pela API de gerenciamento da Supabase (só a listagem de projetos, sem conexão com o banco): `zondscnfosubxutpshux gc-prod INACTIVE`, e `gzbmhgnpoehormnidmgg gc-dev ACTIVE_HEALTHY`, os dois em Postgres 17.6.1.105.

O ensaio rodou sobre a planilha falsa (`scripts/importacao/planilha-falsa.mjs`), porque a "Gestão Contratos.xlsx" real ainda não chegou. As decisões do 14.1 (contrato por obra, aditivos, status da obra, corte do FD) estão aplicadas no importador e foram exercitadas aqui. **Estado do bloco: ensaio executado e passando; a virada em si (prod) é registro.** A rodada completa do `npm run validar` fica com o fechamento da sprint.

1. As atividades do card

ATIVIDADE	ESTADO	EVIDÊNCIA (GC-DEV, 2026-10-26)
Backup do gc-prod	**executado em gc-dev (ensaio).** Em prod: registro	`npx supabase db dump --linked` (schema, 21:55, 1 min 37 s, 163.656 bytes) e `--data-only --use-copy` (21:57, 17,7 s, 3.071.438 bytes), em `~/.claude/jobs/2a37dcc8/tmp/backup-gcdev/`, fora do repositório. O link conferido: `supabase/.temp/project-ref` = `gzbmhgnpoehormnidmgg`. Lido de volta pela restauração (14.5, seção 1)
Aplicar migrations pendentes	**executado em gc-dev (ensaio): nada pendente.** Em prod: registro (46 do runbook, hoje 47 locais)	`npx supabase migration list --linked`: 47 locais e 47 remotas, nenhuma só de um lado, inclusive `20261019100000_pedidos_acesso` e `20261026100000_progresso_itens_so_contratado`. Em gc-dev o repair já tinha sido feito (4.1)

ATIVIDADE	ESTADO	EVIDÊNCIA (GC-DEV, 2026-10-26)
Rodar a importação	**executado em gc-dev (ensaio)**, planilha falsa, `--ate 2026-11-12`. Em prod: registro (o `gc-dev-guard` bloqueia)	simulação: 2 clientes, 3 obras, 4 propostas, 4 itens, 2 contratos, 3 itens de contrato, 1 aditivo, 4 execuções, 2 NFs, 2 pagamentos, 2 FDs; 4 linhas ignoradas e 15 avisos (esperados da planilha falsa). `--gravar` (21:58:43, 2,8 s): tudo criado nessas contagens
Conciliar de novo	**executado em gc-dev (ensaio)**	`conciliar.mjs --sem-referencia`: antes de gravar, 0/38 (prova de que ela acusa); depois: **41/41 linhas ok, 0 divergentes, 0 avisos, status de NF 2/2** (contagens 9/9, totais 3/3, por obra 9/9, propostas 11/11, views 9/9). Segunda gravação: **0 criados** em todas as 11 entidades (clientes 2, obras 3, propostas 4, itens 4, contratos 2, itens de contrato 3, aditivos 1, execuções 4, NFs 2, pagamentos 2, FDs 2 já existiam)
Criar usuários reais	**não executado, por regra:** o fluxo de convite por link já é provado pela camada escrita; prod é registro	ver seção 1.1
Congelar a planilha	**registro:** só faz sentido na virada, com a planilha real, depois de a conciliação fechar	runbook, seção 9

Depois do ensaio: `importar-planilha.mjs --desfazer` apagou 2 FDs, 2 pagamentos, 2 NFs, 4 execuções, 3 itens de contrato, 1 aditivo, 2 contratos, 4 itens, 4 propostas, 3 obras e 2 clientes (a ordem do 14.1). `verificar-sobras.mjs`: "sem sobras" antes e depois. Contagens de gc-dev iguais às de antes do ensaio (obras 4, clientes 57, propostas 17, contratos 10, itens 76). `baixar-drive.mjs` rodou só em **simulação** (2 links: 1 proposta, 1 pagamento; não baixa, não conecta), então não houve nada a desfazer no Storage.

1.1 Usuários

Nada foi criado. O fluxo de convite por link (admin convida, o link `verifyOtp` define a senha) já é exercitado pela camada **escrita** do `npm run validar`, e o `meu-perfil` / `esqueci-minha-senha` têm entrega própria. Regra decidida no 14.1: **2 admins**. Para a virada em prod: o primeiro admin entra pelo painel do Supabase (o convite exige admin logado); o segundo admin e os demais, por convite por link (runbook, seção 7). A lista real de usuários (nome, e-mail, telefone, perfil) não chegou.

1.2 Limites do ensaio

- A planilha é falsa. Os totais do card (referência da conciliação) são da planilha real, então a conciliação rodou com `--sem-referencia`. A aba Resumo da falsa não é lida, e a view foi comparada com os registros montados (a saída avisa).
- A fumaça e a importação em gc-dev não provam o desempenho nem o volume de prod.

2. O que a leitura do repositório encontrou

Levantado para o runbook, sem executar nada:

1. **46 migrations** em `supabase/migrations/`. O histórico da CLI em gc-prod nunca foi reparado. A hipótese é que as 36 da 011 em diante não estão lá, porque foram criadas a partir de 2026-09-05, quando o gc-prod já estava `INACTIVE` (registro do 4.1). O estado da 001 à 010 se descobre com `supabase/verificar_schema_aplicado.sql`.
2. **Uma destrutiva:** a `20260511151924_revisao_schema.sql` (ex-004) apaga 13 colunas de `obras` e 6 de `orcamentos`. Duas mexem em dado existente (`contatos_canal`, `execucao_medicoes`), sete criam constraint que valida as linhas existentes, e 14 arquivos não têm `begin; … commit;`.
3. **Três corrigem falhas de segurança que existem em gc-prod** (13.1 e 13.3): `20261005120000`, `20261007100000` e `20261007110000`. O cadastro aberto do Auth também precisa ser desligado lá (`disable_signup`).
4. **A FK do aditivo** (`20261012110000`) só pode entrar com o código da `main` publicado. A ordem fica: sistema parado → migrations → deploy → fumaça.
5. **A migration inicial cria a empresa "SUA EMPRESA AQUI LTDA".** O primeiro admin de prod entra pelo painel, porque o convite exige um admin já logado.
6. **A importação em prod precisa de `--autor`:** o padrão é o usuário da validação, que não existe em prod.
7. **A fumaça do repositório não roda em prod:** ela cria `VALIDA-FUMACA-*` e usa os perfis de teste. O runbook traz uma fumaça manual, sem gravação.
8. **O deploy que o n8n chama aponta para o gc-dev.** O `gc-sistema-nine.vercel.app` (fork) é ligado ao gc-dev. Os documentos citam outro deploy, do repositório principal, apontando para gc-prod.

3. O que a equipe precisa decidir ou fornecer

1. ~~Autorização do Breno para gc-prod~~ — gc-prod fora do escopo por decisão do Breno (26/10): o bloco fecha em gc-dev; o restante é registro para quem for operar prod. Registro: quem for operar prod precisa dela e de quem opera.
2. **Acesso de dono à conta da Supabase** para reativar o gc-prod. Confirmar no painel se o *Restore* ainda está disponível depois de tanto tempo pausado. Se não estiver, decidir entre restaurar o backup num projeto novo e começar do zero.
3. **A data da janela de virada.** Decidido no 14.1: alvo sexta 13/11/2026, com `--ate 2026-11-12`; reserva 27/11 (corte 26/11). Vale para quem operar prod.
4. **A planilha real e as decisões do 14.1** (o 14.2 e o 14.3 dependem delas).
5. **A lista de usuários:** nome, e-mail, telefone e perfil.
6. **Os dados da empresa** (nome e CNPJ) para trocar o "SUA EMPRESA AQUI LTDA".
7. **Qual projeto da Vercel vira produção**, e se a automação (n8n e bot do Telegram) vai para prod no go-live ou depois.
8. **Os valores de referência** da conciliação continuam valendo para a data de corte?

9. **A liberação do guard** (runbook, seção 6): aprovar o desenho antes de alguém escrever o código.

4. Validação

Este bloco **não mudou código**; só scripts já existentes foram executados e os documentos editados. Números reais do ensaio de 2026-10-26:

CAMADA / COMANDO	SAÍDA
`npx supabase db dump --linked` e `--data-only --use-copy`	schema 163.656 bytes (1 min 37 s); dados 3.071.438 bytes (17,7 s)
restauração local (14.5, seção 1)	20/20 tabelas de `public` com contagem igual à de gc-dev
`npx supabase migration list --linked`	47 locais, 47 remotas, 0 pendentes
`planilha-falsa.mjs` + importação em simulação	2 clientes, 3 obras, 4 propostas, 4 itens, 2 contratos, 3 itens de contrato, 1 aditivo, 4 execuções, 2 NFs, 2 pagamentos, 2 FDs
`--gravar` / `--gravar` de novo	tudo criado / 0 criados (11 entidades)
`conciliar.mjs --sem-referencia`	41/41 linhas ok, 0 divergentes, status de NF 2/2
`--desfazer` + `verificar-sobras.mjs`	21 registros desfeitos nas 11 entidades; sem sobras
`bash scripts/validacao/validar.sh build fumaca`	build ok (56 s, 66 rotas, iguais à baseline); **fumaça 22/22 passos ok** (44 s)

A rodada completa de 2026-10-27 (`npm run validar`) passou nas sete camadas: unitário 432, build 66 rotas, runtime 327/327, dados 65/65, escrita 529/529 (ensaio 17/17), navegador 281/281.

Provados antes, na leitura: `ls supabase/migrations | wc -l` (47 hoje), `GET /v1/projects` (gc-prod `INACTIVE`, gc-dev `ACTIVE_HEALTHY`), `grep process.env src` (as 9 variáveis da seção 5 do runbook).

5. Pendências

- **Virada em prod: só registro.** Backup, repair, as migrations (a destrutiva `20260511151924_revisao_schema`), importação com `--autor`, usuários reais, fumaça manual e o congelamento da planilha seguem o runbook, para quem operar prod.
- **Importação real:** depende da "Gestão Contratos.xlsx". O ensaio com a planilha real deve repetir simulação, gravação, conciliação com a referência do card, segunda gravação (0 criados) e, se for ensaio, o desfazer.
- **Conferência visual das obras importadas pela aplicação** não foi feita (opcional no ensaio).
- **O guard com liberação para prod** está só descrito (seção 3, item 9). Não é tarefa nossa.

- **O `README.md` da raiz está desatualizado.** A tabela de variáveis lista 3 das 9, e a de ambientes diz que a Vercel publica gc-prod a partir de `main`.